SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA0612	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	**JAYAKRISHNA.V**	**Reg. No.:**	**192421259**
Section / Batch:	**I**	**Date:**	**28.07.2026**

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation. • Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach. • Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q29

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I (JAYAKRISHNA.V / 192421259) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JAYAKRISHNA.V

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				

Total Marks (100):	

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q29. Banking System – Binary Search

Problem Statement

A sorted dataset is used in a banking system for quick balance lookup using Binary Search, but frequent updates disturb order.

a) Explain Binary Search operation.

Since the customer account records are arranged in ascending order, the banking system can use Binary Search to locate a customer's balance quickly. Binary Search works by repeatedly dividing the dataset into two equal halves and comparing the target account number with the middle element.

If the middle element matches the target, the search stops. If the target is smaller, the algorithm continues searching in the left half. If the target is larger, it continues in the right half. This process repeats until the account is found or no elements remain.

Steps of Binary Search

1. Arrange the customer account numbers in sorted order.
2. Find the middle element of the dataset.
3. Compare the middle account number with the target account.
4. If both are equal, return the customer record.
5. If the target is smaller, search the left half.
6. If the target is larger, search the right half.
7. Repeat until the account is found or the search interval becomes empty.

Example

Customer Account Numbers

[1012, 1025, 1040, 1058, 1075, 1092, 1108]

Target Account Number

1075

Search Process

Step	Search Range	Middle Account	Comparison	Result
1	1012–1108	1058	1075 > 1058	Search Right Half

Step	Search Range	Middle Account	Comparison	Result
2	1075–1108	1092	1075 < 1092	Search Left Half
3	1075	1075	Match	Account Found

The required customer account is found after 3 comparisons, demonstrating the efficiency of Binary Search.

Example

Customer Account Numbers

[1012, 1025, 1040, 1058, 1075, 1092, 1108]

Target Account Number

1075

Search Process

Step	Search Range	Middle Account	Comparison	Result
1	1012–1108	1058	1075 > 1058	Search Right Half
2	1075–1108	1092	1075 < 1092	Search Left Half
3	1075	1075	Match	Account Found

The required customer account is found after 3 comparisons, demonstrating the efficiency of Binary Search.

---

b) Analyze performance when order is maintained vs disturbed.

Performance When Order is Maintained

When all customer records remain sorted, Binary Search can be applied directly. Every comparison reduces the search space by half, resulting in very fast search performance.

Time Complexity

Case	Description	Time Complexity
Best Case	Target is the middle element	O(1)
Average Case	Search space reduces by half repeatedly	O(log n)

Case	Description	Time Complexity
Worst Case	Target is at the deepest level or absent	O(log n)

Space Complexity

Binary Search uses only a few extra variables.

Auxiliary Space = O(1)

Performance When Order is Disturbed

Frequent insertions, deletions, or updates disturb the sorted order of the banking records. Once the dataset becomes unsorted, Binary Search cannot be applied directly.

The system has two possible choices:

- Sort the dataset again before searching.
- Use Linear Search until the dataset is sorted.

Performance Comparison

Situation	Algorithm Used	Time Complexity
Dataset is Sorted	Binary Search	O(log n)
Dataset is Unsorted	Linear Search	O(n)
Re-sorting Dataset	Efficient Sorting Algorithm	O(n log n)

Therefore, disturbing the sorted order increases the overall search time and reduces system efficiency.

---

c) Discuss importance of sorted structure.

Maintaining a sorted structure is very important in a banking system because Binary Search depends entirely on sorted data.

Advantages of Sorted Structure

- Enables Binary Search.
- Reduces search time from O(n) to O(log n).
- Provides faster customer balance lookup.

- Improves performance for large banking databases.
- Increases efficiency when thousands of customer records are searched every

Disadvantages of Frequent Updates

- New customer accounts disturb the sorted order.
- Deleting records also affects the arrangement.
- Updating account information may require repositioning records.
- The dataset must be sorted again before Binary Search can be used.
- Maintaining sorted order introduces additional processing time.

Justification

In a banking system, customers frequently perform balance inquiries, making fast searching essential. A sorted dataset allows Binary Search to retrieve account information quickly with O(log n) time complexity. However, frequent updates disturb the sorted order, making Binary Search ineffective until the records are reorganized. Therefore, maintaining a sorted structure is highly beneficial when searches occur more frequently than updates, while frequent updates increase maintenance overhead.

Final Answer

a) Binary Search locates a customer's account by repeatedly dividing a sorted dataset into two halves and comparing the middle element with the target account number until the account is found.

b)

Situation	Time Complexity
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)
Space Complexity	O(1)
Disturbed Order (Linear Search)	O(n)

Situation	Time Complexity
Re-sorting Dataset	O(n log n)

c)

A sorted structure is essential because it enables Binary Search, reduces search time, improves banking system performance, and allows quick balance lookup. Although frequent updates disturb the order and require additional sorting, maintaining sorted data provides significant benefits for systems where searches are performed more often than updates.